Matière : **Conception et développement d'applications**
Thème : **Préparation épreuve E6 BTS SIO option SLAM**
Application 1 : **Cahier des charges techniques**

Rédigé par : Yuung Addi KANSIETOKO, BTS SIO option SLAM, Lycée La Tournelle (La Garenne-Colombes)

Version : 1.0 du 12 octobre 2026

Sommaire

1. Contexte du projet....2
1.1. Présentation du projet....2
2. Besoins fonctionnels....2
3. Ressources nécessaires à la réalisation du projet....3
3.1. Ressources matérielles....3
3.2. Ressources logicielles....3
3.3. Ressources documentaires....3
4. Gestion du projet....3
5. Conception du projet....5
5.1. Le front-end....5
5.1.1. Wireframes....5
5.1.2. Maquettes....8
5.1.3. Arborescences....12
5.2. Le back-end....12
5.2.1. Diagramme de cas d'utilisation....12
5.2.2. Diagramme d'activités....13
5.2.3. Modèle Conceptuel de Données (MCD)....13
5.2.4. Modèle Logique de Données (MLD)....14
5.2.5. Modèle Physique de Données (MPD)....14
6. Technologies utilisées....15
6.1. Langages de développement Web....15
6.2. Base de données....15
7. Sécurité....16
7.1. Login et protection des pages administrateurs....16
7.2. Hachage des mots de passe avec Bcrypt....16
7.3. Protection contre les attaques XSS (Cross-Site Scripting)....16
7.4. Protection contre les injections SQL....17
7.5. Mesures complémentaires et conformité RGPD....17

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

Deux profils d'utilisateurs sont pris en compte :

- **Usager** (organisateurs, médias, spectateurs) : consultation libre, sans authentification.
- **Administrateur** : accès authentifié à un back-office qui couvre l'ensemble des données publiées.

Périmètre public

Page	Fonction
Accueil	Point d'entrée, accès direct aux trois rubriques de consultation
Sports	Liste des disciplines, triée par ordre alphabétique
Calendrier des épreuves	Épreuves avec sport, date, heure et lieu
Résultats	Résultat par épreuve : sport, athlète, pays, performance
Accès administrateur	Authentification par login et mot de passe

Périmètre administrateur

Chaque rubrique implémente le CRUD complet (création, lecture, mise à jour, suppression) sur l'entité correspondante.

Rubrique	Entité	Données gérées
Administrateurs	ADMINISTRATEUR	Nom, prénom, login, mot de passe (empreinte Bcrypt)
Sports	SPORT	Libellé du sport
Lieux	LIEU	Nom, adresse, code postal, ville
Pays	PAYS	Libellé du pays
Calendrier	EPREUVE	Nom, date, heure, lieu (FK), sport (FK)
Athlètes	ATHLETE	Nom, prénom, pays (FK), genre (FK)
Genres	GENRE	Libellé du genre
Résultats	PARTICIPER	Résultat d'un athlète sur une épreuve (clé composée)

Règles de gestion et exigences non fonctionnelles

- Contrôle d'unicité à la création et à la modification : aucun doublon n'est accepté (libellé, login, couple athlète/épreuve).
- Validation systématique des saisies côté serveur avant toute écriture en base.
- Back-office inaccessible sans session authentifiée : redirection immédiate vers l'accueil.
- Interface responsive sur trois formats cibles : ordinateur, tablette, smartphone.

3. Ressources nécessaires à la réalisation du projet

Ressources mobilisées de la conception à la mise en production.

3.1. Ressources matérielles

- Poste de développement (portable ou fixe), 8 Go de RAM minimum pour faire tourner simultanément l'IDE, la pile Apache/MySQL locale et le navigateur.
- Accès Internet pour GitHub, Trello, Figma, la documentation et le déploiement.
- Smartphone et tablette pour la recette responsive sur terminaux réels, en complément de l'émulation des DevTools.
- Hébergement mutualisé **InfinityFree** (offre gratuite : Apache, PHP, MySQL) pour l'environnement de production.

3.2. Ressources logicielles

Besoin	Outil retenu	Usage
IDE	Visual Studio Code	Développement HTML, CSS, PHP, JS, SQL ; extension PHP Intelephense
Environnement local	MAMP (Apache, MySQL, PHP 8)	Exécution du site et de la base en local
Administration BDD	phpMyAdmin	Création de la base, import des scripts, contrôle des données
Navigateurs	Chrome, Firefox	Recette fonctionnelle, inspection, émulation responsive
Wireframes	Figma	Wireframes ordinateur, tablette, mobile
Modélisation données	Mocodo	MCD, MLD, génération du script SQL
Modélisation UML	Visual Paradigm Online	Cas d'utilisation, diagramme d'activités
Pilotage	Trello	Tableau Kanban, échéancier
Gestion de versions	Git, GitHub	Historique, sauvegarde, livraison du code et du CDC
Tests unitaires	Composer, PHPUnit	Tests automatisés des opérations CRUD
Déploiement	InfinityFree, FileZilla (FTP)	Hébergement de la base et transfert des fichiers

3.3. Ressources documentaires

- **php.net** : référence PHP (PDO, password_hash, htmlspecialchars, sessions).
- **MDN Web Docs** : référence HTML, CSS, JavaScript.
- **OWASP** : Top 10, fiches XSS, injection SQL, gestion de session.
- **CNIL** : obligations RGPD sur les données à caractère personnel.
- **Documentation PHPUnit** et **documentation MySQL 8**.
- **W3Schools** : exemples rapides de syntaxe.
- Supports de cours BTS SIO et module de gestion des sports fourni par le lead developer.

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.

Le tableau Trello est organisé en trois colonnes (**À faire**, **En cours**, **Terminé**). Une carte correspond à un livrable ; chaque carte porte une échéance et une étiquette de nature : Conception, Développement, Sécurité, Tests, Déploiement, Administratif. La colonne « En cours » est limitée à une ou deux cartes pour éviter de disperser l'effort.

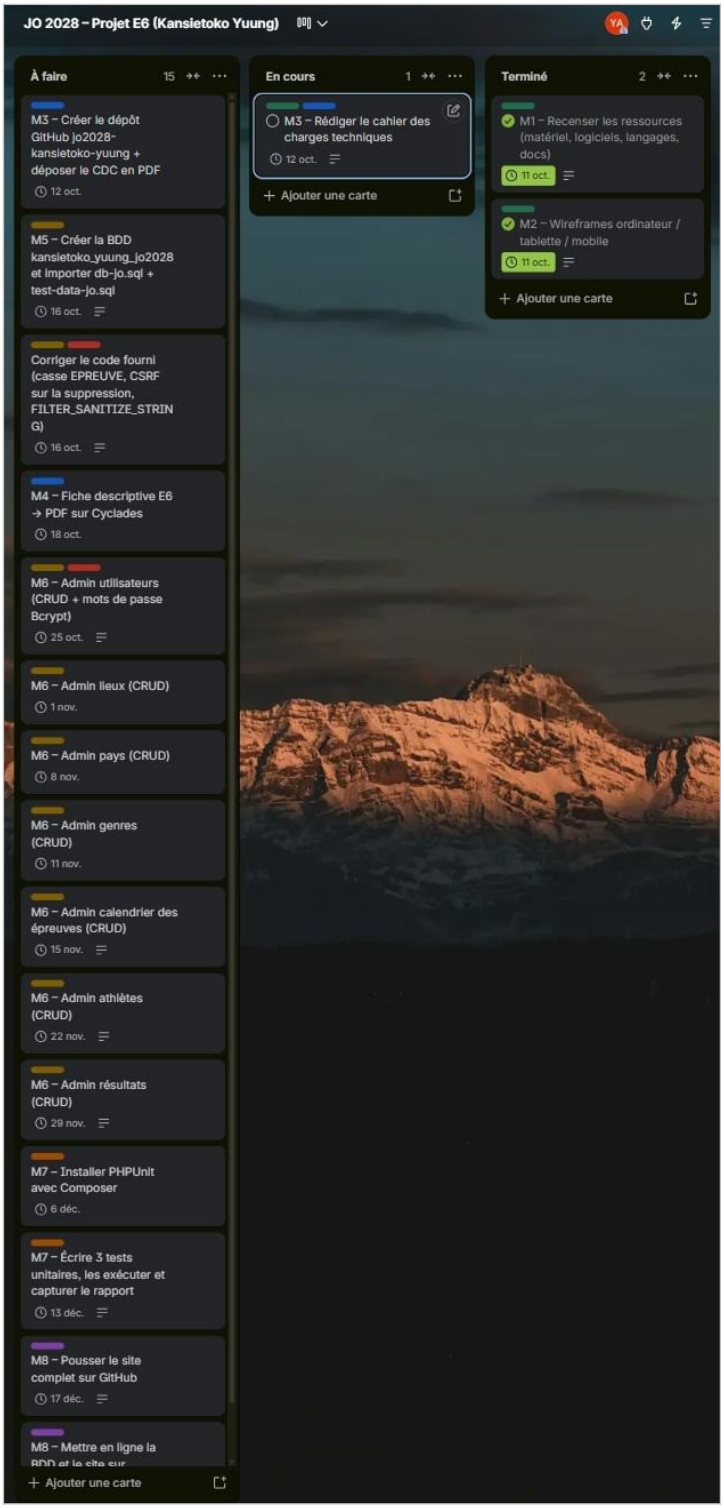


Tableau Kanban du projet (Trello), état au 11 octobre 2026

Jalons

Échéance	Livrable	Mission
11/10/2026	Recensement des ressources, wireframes	M1, M2
12/10/2026	Cahier des charges techniques en PDF sur GitHub	M3
16/10/2026	Base de données opérationnelle, correctifs du code fourni	M5
18/10/2026	Fiche descriptive E6 déposée sur Cyclades	M4
25/10 au 29/11/2026	Back-office : 7 rubriques CRUD, une par semaine	M6
13/12/2026	Tests unitaires PHPUnit et rapport d'exécution	M7

Échéance	Livrable	Mission
20/12/2026	Mise en production InfinityFree, code complet sur GitHub	M8

Nous travaillons également sur GitHub, plateforme de développement collaboratif.

Dépôt : **jo2028-kansietoko-yuung**. Il contient le cahier des charges (PDF) et le code source. Un commit par fonctionnalité livrée, avec un message explicite. Les identifiants de connexion à la base sont exclus du dépôt.

5. Conception du projet

5.1. Le front-end

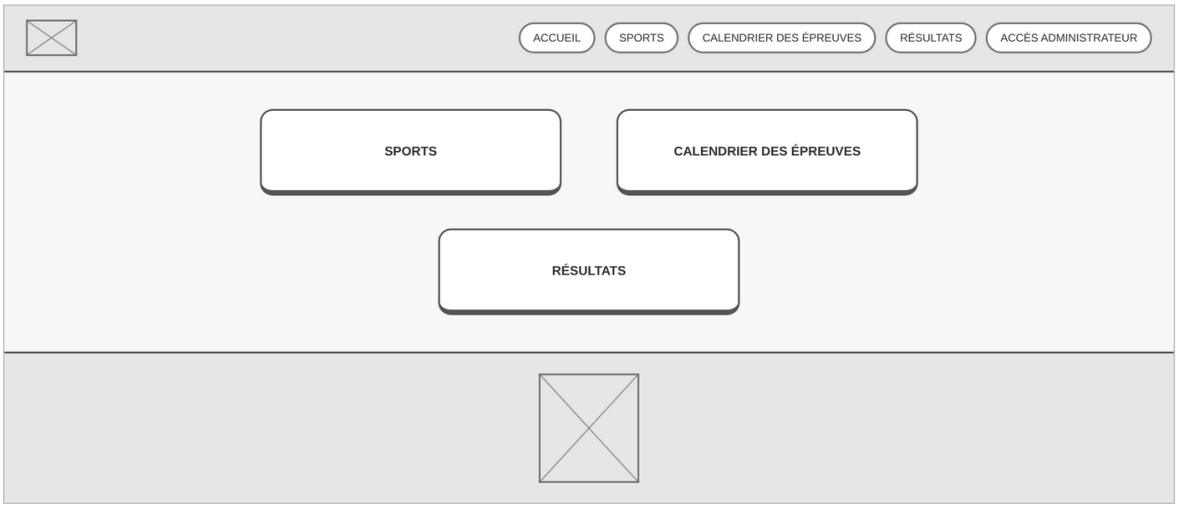
Démarche en trois temps : arborescence (pages et navigation), wireframes (structure et hiérarchie de l'information), maquettes (charte graphique appliquée).

5.1.1. Wireframes

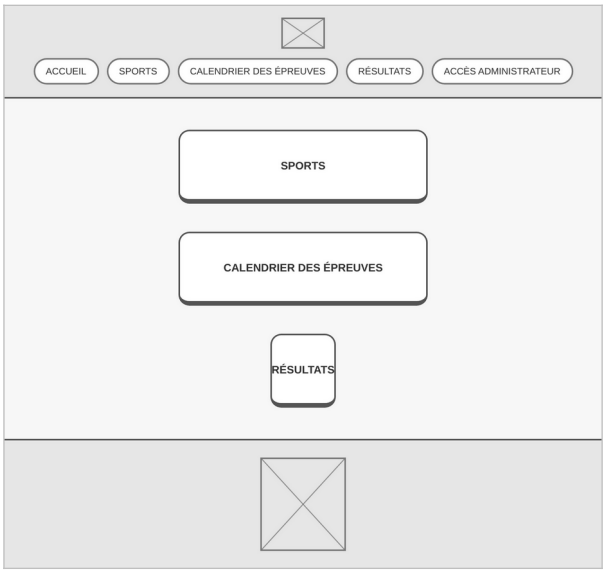
Wireframes basse fidélité des trois gabarits du site : page d'accueil, page de liste (calendrier des épreuves, gabarit réutilisé pour les sports et les résultats) et page de formulaire (connexion, gabarit réutilisé par le back-office).

Points de rupture : au-delà de 900 px, menu horizontal aligné à droite. Entre 480 et 900 px, menu centré sous le logo, cartes d'accueil sur une colonne. En dessous de 480 px, liens de navigation empilés et tableau compacté.

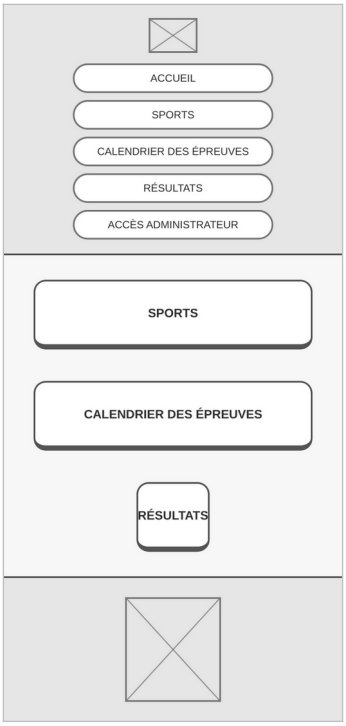
Page d'accueil



Wireframe Accueil, ordinateur



Accueil, tablette



Accueil, smartphone

Page « Calendrier des épreuves »



Wireframe Calendrier des épreuves, ordinateur

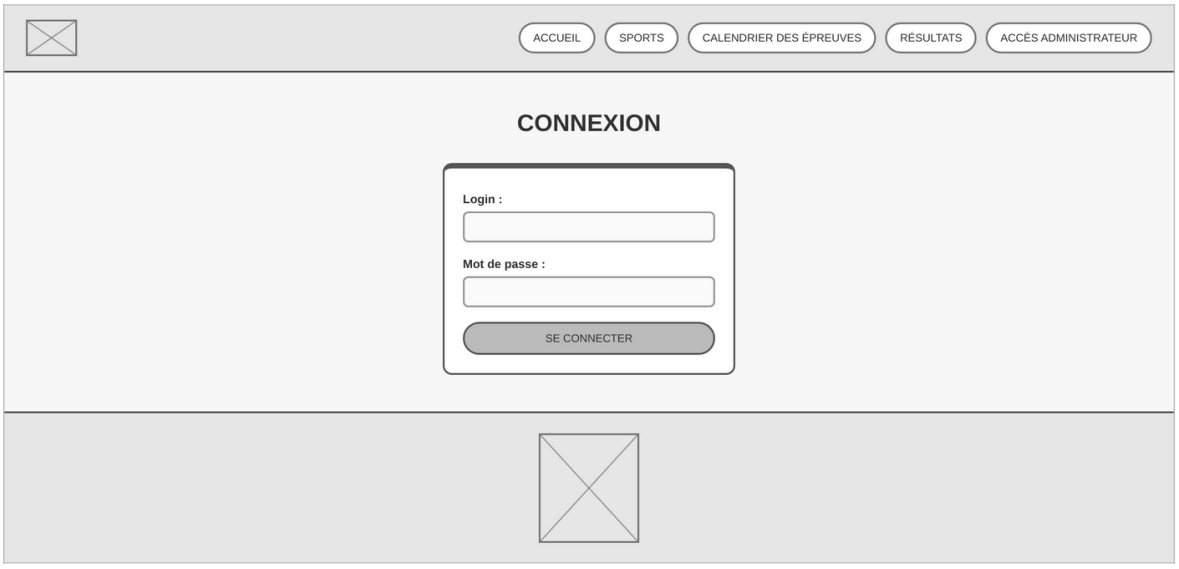


Calendrier, tablette

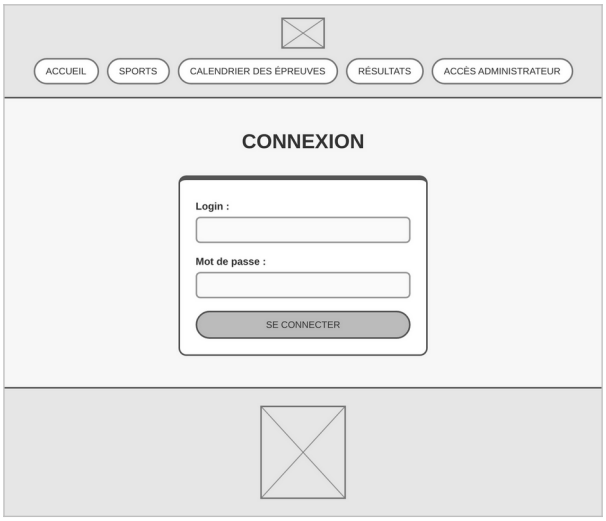


Calendrier, smartphone

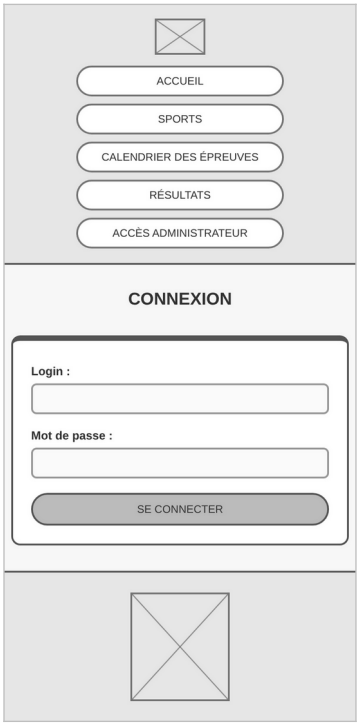
Page « Connexion »



Wireframe Connexion, ordinateur



Connexion, tablette



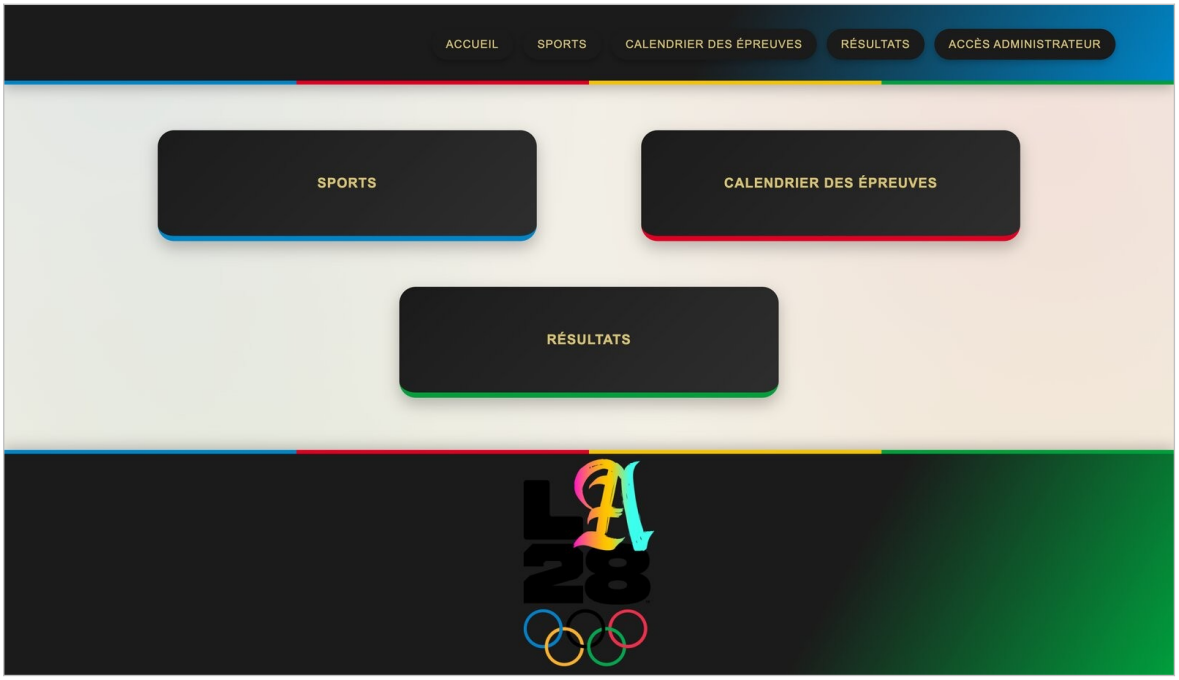
Connexion, smartphone

5.1.2. Maquettes

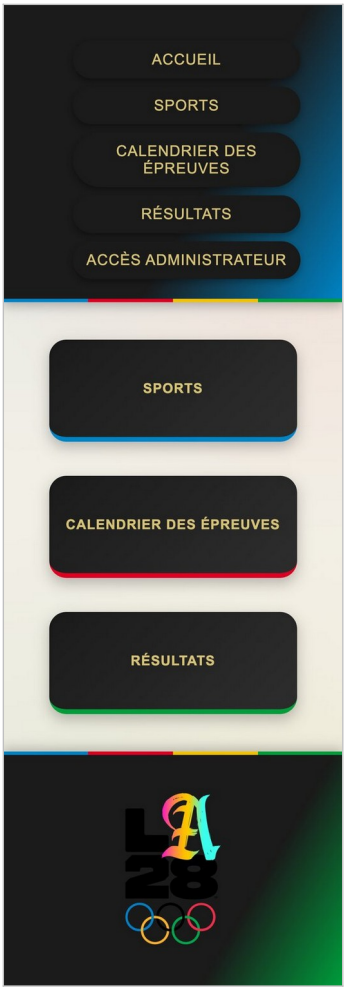
Charte graphique retenue, alignée sur l'identité LA28 :

- en-tête et pied de page sur fond #1B1B1B, dégradé vers le bleu (haut) et le vert (bas) ;
- liseré quadricolore reprenant les anneaux olympiques comme séparateur ;
- typographie dorée sur les éléments de navigation, boutons arrondis avec ombre portée ;
- logo officiel LA28 en pied de page ;
- tableaux à en-tête en dégradé et lignes alternées.

Page d'accueil

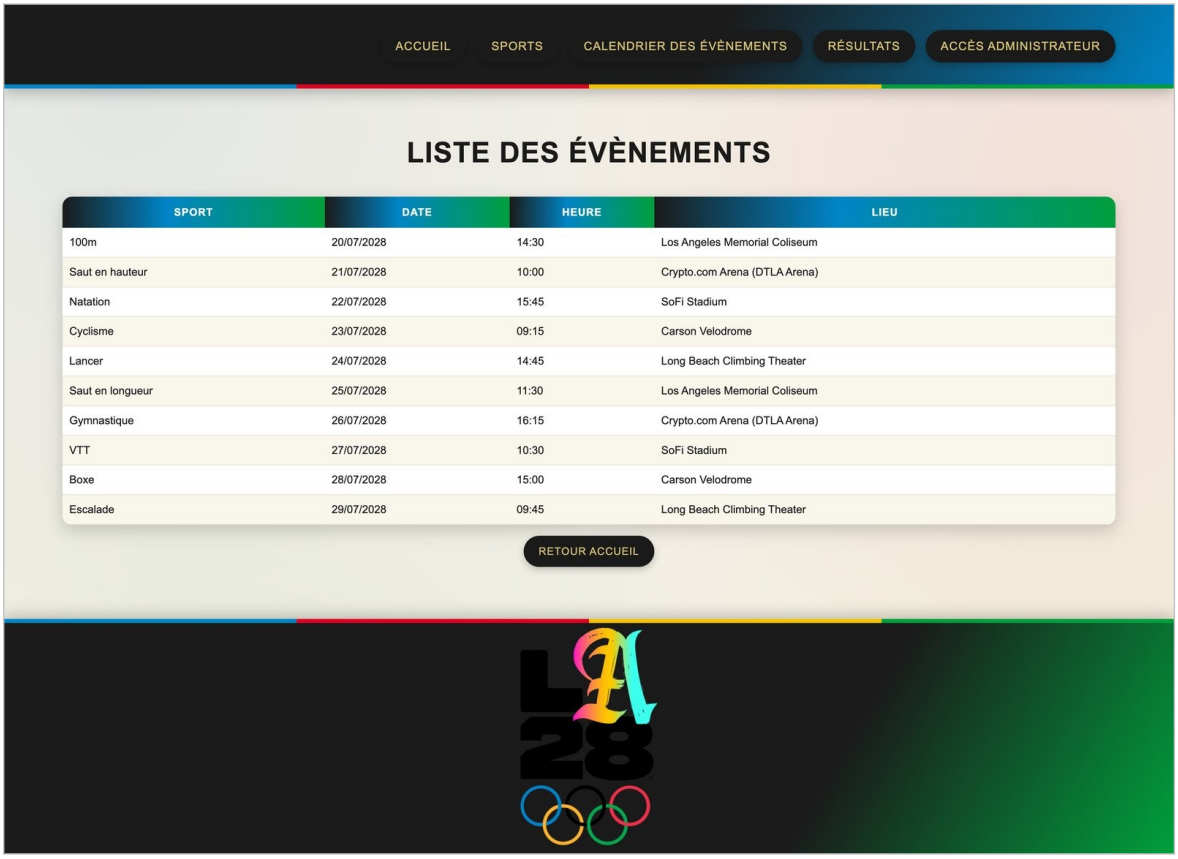


Maquette Accueil, ordinateur



Maquette Accueil, smartphone

Page « Calendrier des épreuves »



Maquette Calendrier des épreuves, ordinateur

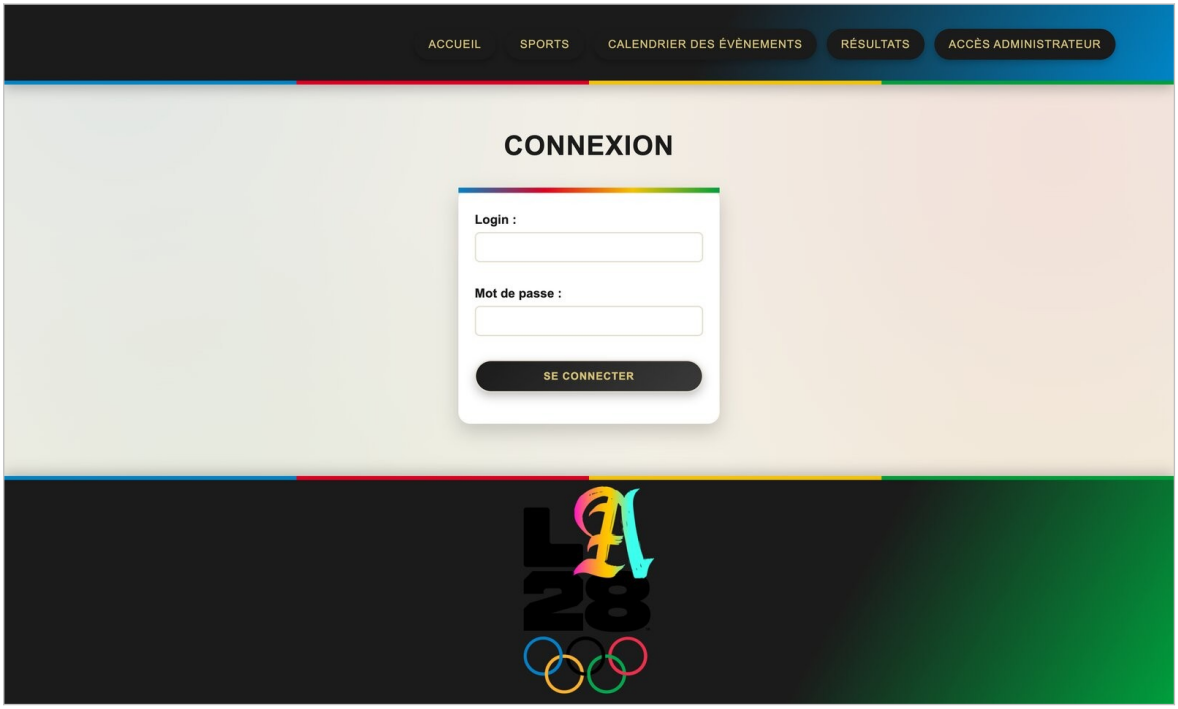


Calendrier, tablette



Calendrier, smartphone

Page « Connexion »



Maquette Connexion, ordinateur

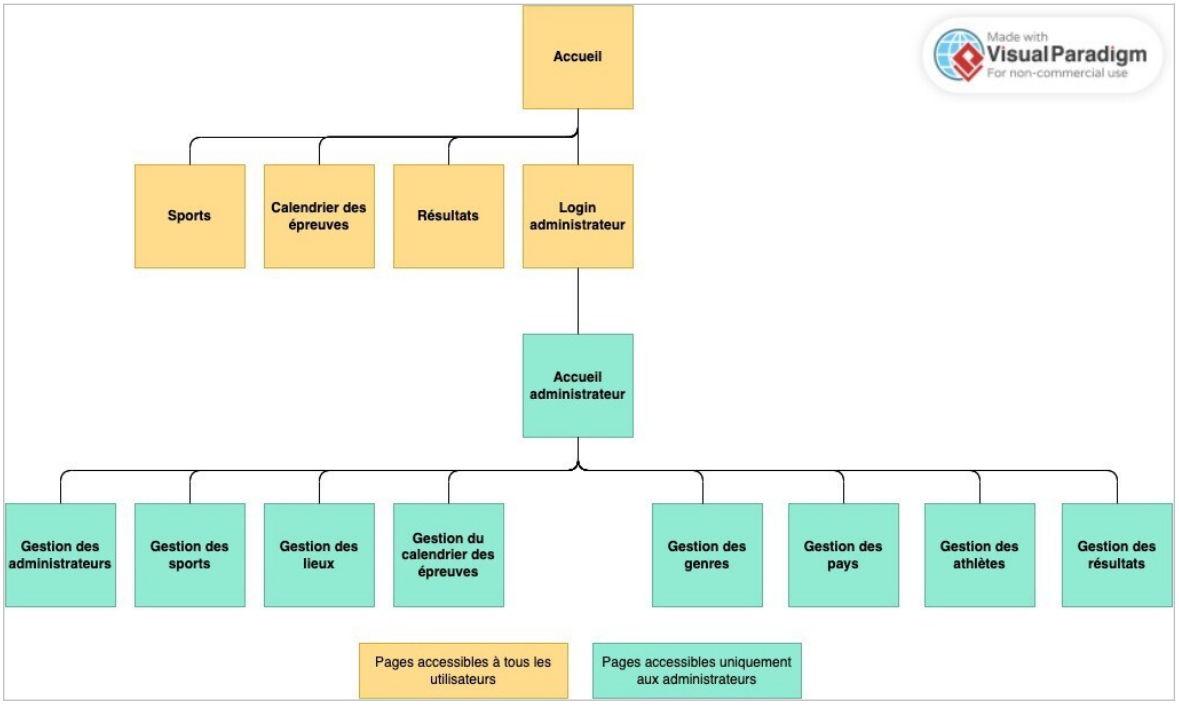


Maquette Connexion, smartphone

Harmonisation : les maquettes emploient indifféremment « Calendrier des épreuves » et « Calendrier des évènements ». Le libellé retenu est **« Calendrier des épreuves »**, conforme à l'arborescence.

5.1.3. Arborescences

Deux niveaux d'accès : pages publiques (orange) et pages du back-office (vert), ces dernières accessibles uniquement depuis l'accueil administrateur après authentification.



Arborescence du site JO 2028

5.2. Le back-end

Comportement modélisé en UML, données modélisées selon la méthode Merise.

5.2.1. Diagramme de cas d'utilisation

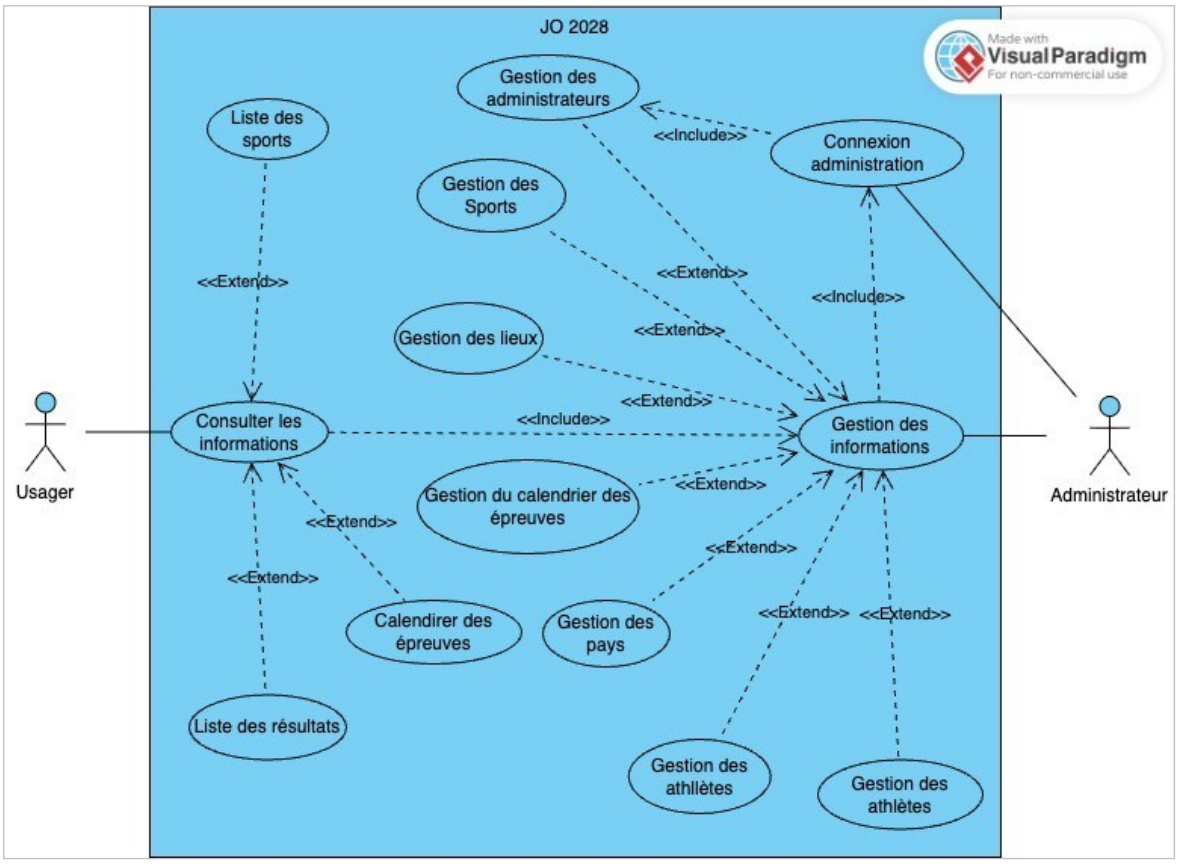


Diagramme de cas d'utilisation

- **Usager** : « Consulter les informations », étendu par la liste des sports, le calendrier des épreuves et la liste des résultats.
- **Administrateur** : « Gestion des informations », qui inclut « Connexion administration » et est étendu par les huit rubriques de gestion.

Anomalie relevée sur le diagramme fourni : « Gestion des athlètes » figure deux fois et « Gestion des genres » est absente. L'occurrence en double est interprétée comme la gestion des genres, présente dans l'arborescence et dans le schéma de base.

5.2.2. Diagramme d'activités

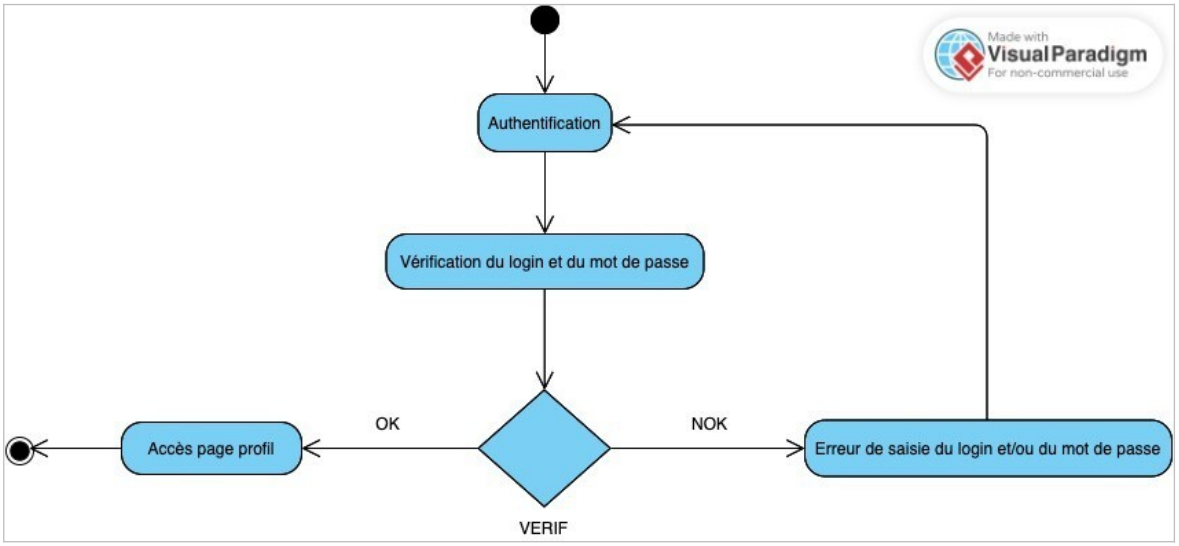
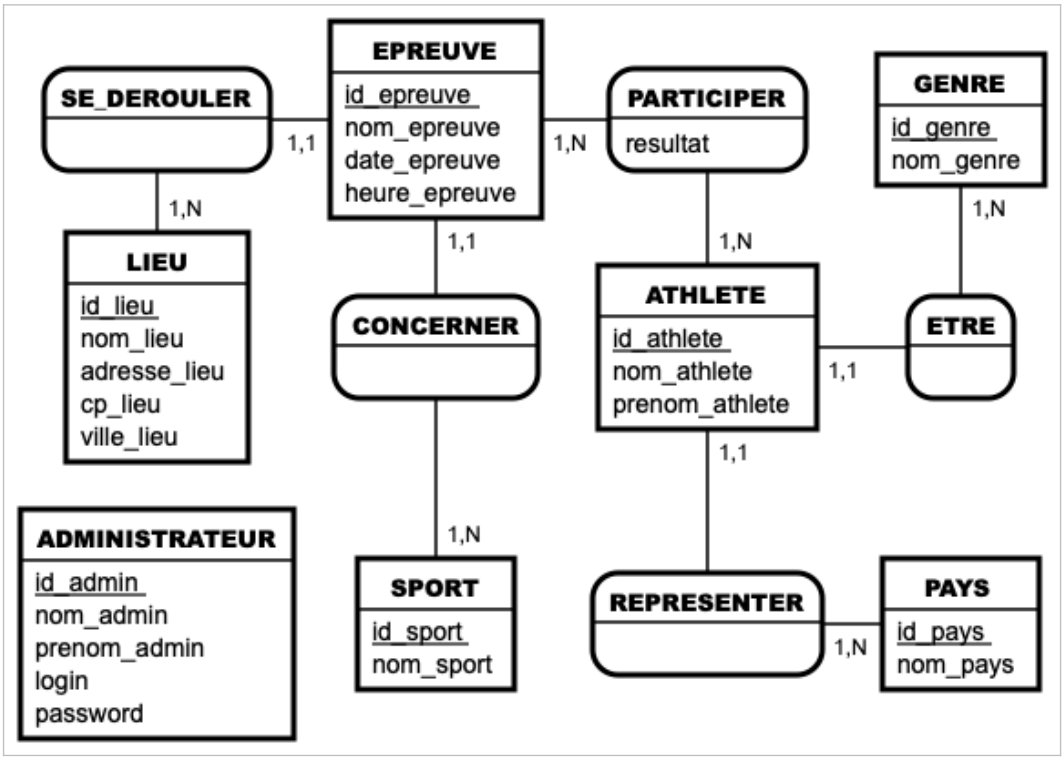


Diagramme d'activités : authentification administrateur

Saisie des identifiants, recherche du login en base, comparaison du mot de passe avec l'empreinte Bcrypt stockée. Succès : ouverture de session et accès au back-office. Échec : message d'erreur générique et retour au formulaire.

5.2.3. Modèle Conceptuel de Données (MCD)



Modèle Conceptuel de Données

Association	Cardinalités	Règle de gestion
SE_DEROULER	EPREUVE 1,1 / LIEU 1,N	Une épreuve a lieu sur un seul site
CONCERNER	EPREUVE 1,1 / SPORT 1,N	Une épreuve relève d'un seul sport
PARTICIPER	EPREUVE 1,N / ATHLETE 1,N	Association porteuse de l'attribut resultat
REPRESENTER	ATHLETE 1,1 / PAYS 1,N	Un athlète représente un seul pays
ETRE	ATHLETE 1,1 / GENRE 1,N	Un athlète relève d'un seul genre

ADMINISTRATEUR est une entité isolée, dédiée à l'authentification.

5.2.4. Modèle Logique de Données (MLD)

Syntaxe BTS.

ATHLETE (id_athlete, nom_athlete, prenom_athlete, id_pays, id_genre)
Clé primaire : id_athlete
Clés étrangères : id_pays en référence à id_pays de PAYS
id_genre en référence à id_genre de GENRE

EPREUVE (id_epreuve, nom_epreuve, date_epreuve, heure_epreuve, id_lieu, id_sport)
Clé primaire : id_epreuve
Clés étrangères : id_lieu en référence à id_lieu de LIEU
id_sport en référence à id_sport de SPORT

GENRE (id_genre, nom_genre)
Clé primaire : id_genre

LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)
Clé primaire : id_lieu

PARTICIPER (id_athlete, id_epreuve, resultat)
Clé primaire : id_athlete, id_epreuve
Clés étrangères : id_athlete en référence à id_athlete de ATHLETE
id_epreuve en référence à id_epreuve de EPREUVE

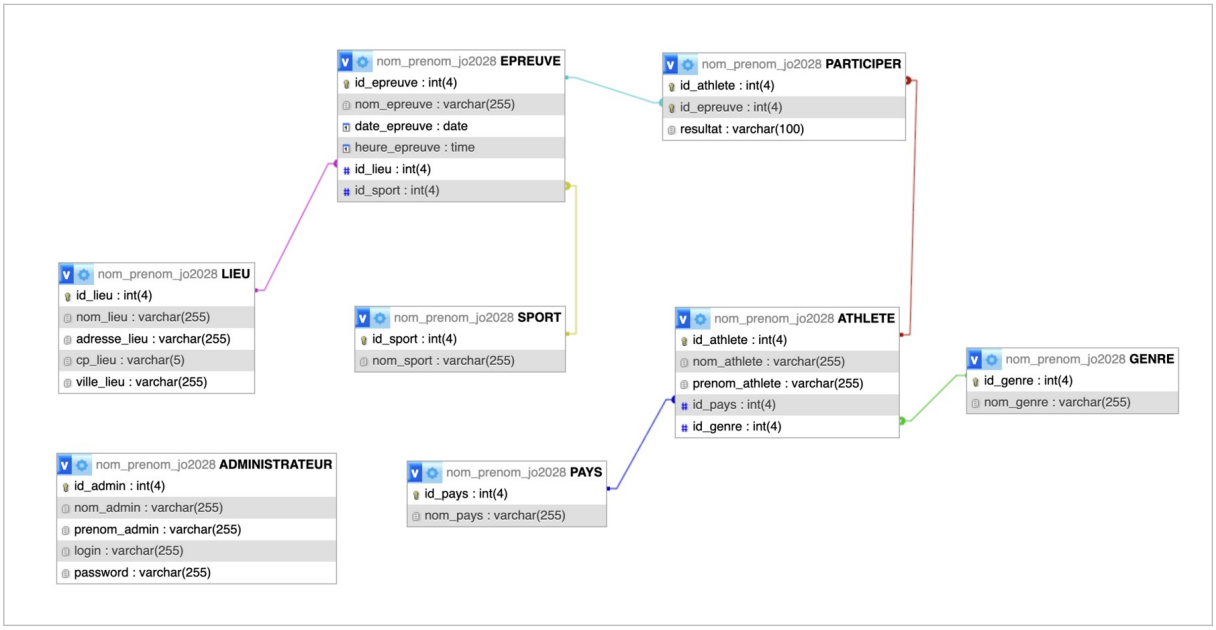
PAYS (id_pays, nom_pays)
Clé primaire : id_pays

SPORT (id_sport, nom_sport)
Clé primaire : id_sport

ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)
Clé primaire : id_admin

Écarts corrigés par rapport au MLD « version classique » fourni : relation RESULTAT(score) renommée PARTICIPER(resultat) pour cohérence avec le MCD et le script SQL ; clé primaire de PAYS ramenée à id_pays ; relation UTILISATEUR supprimée (absente du MCD et du schéma).

5.2.5. Modèle Physique de Données (MPD)



Modèle Physique de Données (phpMyAdmin)

Clés primaires en `INT(4) AUTO_INCREMENT`, chaînes en `VARCHAR`, horodatage des épreuves en `DATE` et `TIME`. Les contraintes de clés étrangères garantissent l'intégrité référentielle : un pays référencé par un athlète ne peut pas être supprimé.

Écart script / modèle : `nom_epreuve` figure au MCD et au MPD mais n'est pas créé par `db-jo.sql`. Correctif appliqué après import :

```
ALTER TABLE EPREUVE ADD nom_epreuve VARCHAR(255) AFTER id_epreuve; -- alignement du schéma sur le MCD
```

6. Technologies utilisées

Choix conformes aux contraintes techniques du commanditaire (annexe 1).

6.1. Langages de développement Web

Langage	Usage
HTML5	Structure sémantique (header, nav, main, footer), formulaires, tableaux
CSS3	Charte graphique ; responsive par media queries (styles-computer.css, styles-responsive.css)
PHP 8	Logique serveur : sessions, authentification, traitement des formulaires, accès aux données via **PDO**
JavaScript	Front-end uniquement, conformément au cahier des charges : ouverture des formulaires, confirmation de suppression
SQL	DDL du schéma, DML via requêtes préparées

6.2. Base de données

SGBD relationnel **MySQL**, moteur InnoDB (support des clés étrangères et des transactions), jeu de caractères utf8mb4.

- **Développement :** MySQL de MAMP, base `kansietoko_yuung_jo2028`, initialisée par `db-jo.sql` (schéma) puis `test-data-jo.sql` (jeu d'essai).
- **Production :** instance MySQL InfinityFree, initialisée par les mêmes scripts.
- **Couche d'accès :** connexion PDO centralisée dans `database/database.php`, mode `ERRMODE_EXCEPTION`, incluse par toutes les pages.

Contrainte de déploiement : sous Linux, MySQL est sensible à la casse des noms de tables. Les requêtes doivent respecter la casse du schéma (`EPREUVE`). La page `events.php` fournie interroge `epreuve` et sera corrigée avant la mise en production.

7. Sécurité

Exigences de l'annexe 1 : mots de passe hachés, back-office protégé, formulaires protégés contre le XSS et l'injection SQL. Le dispositif est complété par une protection CSRF (7.5).

7.1. Login et protection des pages administrateurs

Authentification par formulaire POST traité par `database/auth.php` : recherche du compte par requête préparée, vérification par `password_verify()`, puis ouverture de session. L'identifiant de session est régénéré à la connexion (protection contre la fixation de session). Le message d'échec ne distingue pas login inconnu et mot de passe erroné, afin de ne pas permettre l'énumération des comptes.

```
// database/auth.php
$stmt = $connexion->prepare("SELECT id_admin, nom_admin, prenom_admin, login, password
                             FROM ADMINISTRATEUR WHERE login = :login");
$stmt->bindParam(":login", $login, PDO::PARAM_STR);
$stmt->execute();
$row = $stmt->fetch(PDO::FETCH_ASSOC);
if ($row && password_verify($password, $row["password"])) {
    session_regenerate_id(true);          // anti-fixation de session
    $_SESSION["login"] = $row["login"];   // marqueur d'authentification
    header("Location: ../pages/admin/admin.php");
    exit();
}
```

Garde d'accès en tête de chaque page du back-office :

```
session_start();
if (!isset($_SESSION['login'])) {
    header('Location: ../../../index.php');
    exit();                               // interrompt l'exécution du reste de la page
}
```

Déconnexion : `session_unset()` puis `session_destroy()` dans `logout.php`.

7.2. Hachage des mots de passe avec Bcrypt

Bcrypt est une fonction de hachage dédiée aux mots de passe. Elle intègre un sel aléatoire par empreinte (deux mots de passe identiques produisent deux empreintes distinctes, ce qui neutralise les tables arc-en-ciel) et un facteur de coût paramétrable qui ralentit volontairement le calcul pour rendre la force brute coûteuse. Seule l'empreinte est stockée : une fuite de la table ADMINISTRATEUR n'expose pas les mots de passe en clair.

Implémentation via l'API native de PHP :

```
// création / modification d'un administrateur
$hash = password_hash($password, PASSWORD_BCRYPT);   // coût par défaut : 10
$stmt = $connexion->prepare("INSERT INTO ADMINISTRATEUR (nom_admin, prenom_admin, login,
password)
                             VALUES (:nom, :prenom, :login, :password)");
$stmt->execute([':nom' => $nom, ':prenom' => $prenom, ':login' => $login, ':password' =>
$hash]);

// authentification
if (password_verify($passwordSaisi, $row['password'])) { /* accès accordé */ }
```

Format de l'empreinte : `$2y$10$` + sel + hash (60 caractères). La colonne `VARCHAR(255)` absorbe une éventuelle évolution de l'algorithme (`PASSWORD_DEFAULT`, `password_needs_rehash()`).

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

XSS (Cross-Site Scripting) : injection de code exécutable côté client, généralement du JavaScript, via une donnée ensuite restituée dans une page. Dans ce projet, le risque principal est le XSS stocké : un libellé malveillant enregistré en base s'exécuterait chez chaque visiteur et pourrait servir à détourner la session d'un administrateur.

Contre-mesures :

- **Validation en entrée** : typage et format contrôlés (`filter_input()`, `FILTER_VALIDATE_INT` pour les identifiants, longueur maximale alignée sur le schéma).
- **Encodage en sortie** : toute donnée restituée passe par `htmlspecialchars()` avec `ENT_QUOTES` et l'encodage UTF-8. L'encodage est fait à l'affichage et non à l'enregistrement, pour conserver en base la donnée brute et éviter le double encodage.

```
// pages/sports.php
echo "<td>" . htmlspecialchars($row['nom_sport'], ENT_QUOTES, 'UTF-8') . "</td>";
```

7.4. Protection contre les injections SQL

Injection SQL : altération d'une requête par concaténation d'une saisie non maîtrisée. Exemple : la chaîne `' OR '1'='1` injectée dans une clause WHERE rend la condition toujours vraie et contourne l'authentification. Selon les droits du compte SQL, l'impact va jusqu'à la lecture, la modification ou la suppression de l'intégralité des données.

Contre-mesure : **requêtes préparées PDO** sur 100 % des accès à la base. La structure de la requête est compilée par MySQL avant la transmission des valeurs, qui sont liées comme paramètres typés et ne sont jamais interprétées comme du SQL.

```
// vulnérable (proscrit)
$sql = "SELECT * FROM ADMINISTRATEUR WHERE login = '$login'";

// implémentation retenue
$stmt = $connexion->prepare("DELETE FROM SPORT WHERE id_sport = :id_sport");
$stmt->bindParam(':id_sport', $id_sport, PDO::PARAM_INT);
$stmt->execute();
```

Défense en profondeur : identifiants issus de l'URL validés par `filter_input(INPUT_GET, 'id_sport', FILTER_VALIDATE_INT)` et option `PDO::ATTR_EMULATE_PREPARES` désactivée pour imposer de vraies requêtes préparées côté serveur.

7.5. Mesures complémentaires et conformité RGPD

- **CSRF** : jeton aléatoire (`random_bytes`) stocké en session, embarqué dans chaque formulaire du back-office et vérifié par `hash_equals()`. Les suppressions sont migrées d'un lien GET vers un formulaire POST protégé par ce jeton.
- **Divulgation d'information** : `display_errors` actif en développement uniquement ; en production, les exceptions sont journalisées et un message générique est affiché.
- **Gestion des secrets** : fichier de configuration de la base exclu du dépôt via `.gitignore`.
- **RGPD** : données personnelles limitées au strict nécessaire (identité et login des administrateurs, identité des athlètes), finalité unique de fonctionnement du service, accès restreint aux comptes authentifiés, suppression possible depuis le back-office.